1 Coordinate Frames and Time Systems

Every model in this reference (the tropospheric and ionospheric mappings of the Atmosphere document, the solid-Earth, ocean, and pole tides of the Site Displacement document, the antenna, attitude, and phase-wind-up terms of the Observation Physics document, and the orbit evaluation of the Product Evaluation document) assumes a fixed set of coordinate frames and time scales. This document specifies them in three parts. The first is the WGS84 ellipsoid and the three coordinate representations the engine uses: Cartesian ECEF, geodetic, and local ENU. The second is the GNSS time scales and the leap-second handling that ties them to UTC. The third is the Earth-orientation chain that rotates the celestial frame, in which the Sun, Moon, and satellite ephemerides are expressed, into the terrestrial frame of the receiver. None of these conversions is an estimator state. The one estimated frame quantity is the inter-system time offset (ISB), which absorbs the unmodelled bias between constellation time scales and belongs to the State Vector document. This document covers only the *deterministic* conversions that the ISB is defined relative to. Code references are to `src/core/math/Coordinates.cpp`, `src/core/math/TimeUtils.cpp`, `src/core/model/Planetary.cpp`, `src/core/products/LiveEphemerisStore.cpp`, `src/core/products/ProductParser.cpp`, and `src/core/rinex/RinexParser.cpp`.

1.1 The WGS84 ellipsoid: ECEF, geodetic, and local frames

The terrestrial reference frame is the WGS84 Earth-Centred, Earth-Fixed (ECEF) Cartesian system: origin at the geocentre, z along the conventional pole, x toward the IERS reference meridian, y completing a right-handed triad. The defining ellipsoid has semi-major axis a and flattening f, from which the first eccentricity squared follows (`Coordinates::ecefToLlh` and `Coordinates::llhToEcef` each derive it from f):

$$e^2 \;=\; f\,(2 - f). \tag{1}$$

The radius of curvature in the prime vertical at geodetic latitude φ is

$$N(\varphi) \;=\; \frac{a}{\sqrt{1 - e^2\,\sin^2\varphi}}, \tag{2}$$

which scales the entire forward map from geodetic coordinates (φ, λ, h) to ECEF (`Coordinates::llhToEcef`):

$$\boxed{\begin{aligned} x &= \big(N + h\big)\cos\varphi\,\cos\lambda, \\ y &= \big(N + h\big)\cos\varphi\,\sin\lambda, \\ z &= \big((1 - e^2)\,N + h\big)\sin\varphi. \end{aligned}} \tag{3}$$

The factor $(1 - e^2)$ on the z component is what makes the surface an ellipsoid. On a sphere every axis would carry N unchanged. The factor is also why the geodetic normal, which is perpendicular to the ellipsoid, differs from the geocentric radius vector.

Inverse map (Bowring fixed point). The inverse problem, recovering (φ, λ, h) from (x, y, z), has no closed form for the latitude because N in (3) itself depends on φ. The longitude is immediate, $\lambda = \operatorname{atan2}(y, x)$. For the latitude the engine runs a Bowring fixed-point iteration on the equatorial projection $p = \sqrt{x^2 + y^2}$ (`Coordinates::ecefToLlh`). The seed neglects height,

$$\varphi^{(0)} = \operatorname{atan2}\big(z,\; p\,(1 - e^2)\big), \tag{4}$$

and each pass recomputes N and the height and re-evaluates the latitude with the height feedback folded back into the eccentricity term:

$$h^{(k)} = \frac{p}{\cos\varphi^{(k)}} - N^{(k)}, \qquad \varphi^{(k+1)} = \operatorname{atan2}\!\left(z,\; p\left[1 - e^2\,\frac{N^{(k)}}{N^{(k)} + h^{(k)}}\right]\right). \tag{5}$$

The iteration is capped at ten passes and exits early once $|\varphi^{(k+1)} - \varphi^{(k)}| < 10^{-13}$; for terrestrial heights it converges to machine precision in two or three steps.

Pole branch. When $p < 10^{-12}\,\mathrm{m}$ the point lies on (or within a picometre of) the z axis and both the longitude and the $p/\cos\varphi$ height formula degenerate. The code takes an explicit branch (`Coordinates::ecefToLlh`, the $p < 10^{-12}$ guard): the longitude retains the $\operatorname{atan2}(y, x)$ value computed before the guard, formally undefined on the axis and immaterial there, while the branch fixes $\varphi = \pm\pi/2$ by the sign of z and reads the height directly off the polar semi-minor axis,

$$h = |z| - N\,(1 - e^2) = |z| - a\sqrt{1 - e^2} \quad (\text{at the pole, } \sin^2\varphi = 1), \tag{6}$$

since $N(1 - e^2)$ evaluated at the pole is exactly the polar radius $a\sqrt{1 - e^2}$. This guard keeps the inverse map total over all of $\mathbb{R}^3$, which matters because the planetary and attitude code occasionally evaluates geometry for satellite positions that transit near the polar axis.

Local East–North–Up frame. Many of the downstream models (the tropospheric mapping function, the elevation mask, the antenna phase-centre projection) are most natural in the topocentric ENU frame anchored at the receiver's geodetic (φ, λ). The columns of the ENU→ECEF rotation are the three local unit vectors expressed in ECEF (`Coordinates::enuToEcefMatrix`):

$$\boxed{\boldsymbol{R}_{\mathrm{ENU}}(\varphi,\lambda) = \begin{bmatrix} \hat{\boldsymbol{e}}_{\mathrm{E}} & \hat{\boldsymbol{e}}_{\mathrm{N}} & \hat{\boldsymbol{e}}_{\mathrm{U}} \end{bmatrix} = \begin{bmatrix} -\sin\lambda & -\sin\varphi\cos\lambda & \cos\varphi\cos\lambda \\ \cos\lambda & -\sin\varphi\sin\lambda & \cos\varphi\sin\lambda \\ 0 & \cos\varphi & \sin\varphi \end{bmatrix}.} \tag{7}$$

The Up column is the outward geodetic normal, identical to the direction along which h is measured in (3); East is its horizontal cross-product with the pole, and North completes the right-handed triad. Because the columns are orthonormal, $\boldsymbol{R}_{\mathrm{ENU}}$ is a proper rotation and its inverse is its transpose: an ECEF displacement is resolved into local components by $\boldsymbol{v}_{\mathrm{ENU}} = \boldsymbol{R}_{\mathrm{ENU}}^{\mathrm{T}}\,\boldsymbol{v}_{\mathrm{ECEF}}$ (`Coordinates::ecefToEnu`, with `Coordinates::enuToEcef` the forward direction), avoiding any explicit matrix inversion.

Elevation and azimuth. With the receiver-to-satellite line-of-sight resolved into the local frame, $\boldsymbol{e}_{\mathrm{ENU}} = (e_{\mathrm{E}}, e_{\mathrm{N}}, e_{\mathrm{U}})^{\mathrm{T}} = \boldsymbol{R}_{\mathrm{ENU}}^{\mathrm{T}}\boldsymbol{e}$, the elevation and azimuth that gate the measurement (the elevation mask of the Observation Equations document and the mapping-function argument of the Atmosphere document) follow directly:

$$\mathrm{el} = \arcsin\!\big(e_{\mathrm{U}}\big), \qquad \mathrm{az} = \operatorname{atan2}\!\big(e_{\mathrm{E}},\, e_{\mathrm{N}}\big), \tag{8}$$

with azimuth measured clockwise from North. Equations (7)–(8) connect the ECEF state vector to every elevation-dependent weighting and antenna model in this reference.

1.2 Station reference points: marker, ARP, and phase centre

A station coordinate can refer to any of three points. The *marker* (monument) is the surveyed benchmark, and is what an IGS SINEX/SSC solution, an NGS Station Information File, and the RINEX `APPROX POSITION XYZ` header record all refer to. The *antenna reference point* (ARP) sits where the antenna meets its mount and is the origin against which the ANTEX phase-centre offsets of the Observation Physics document are tabulated. The *antenna phase centre* is where the measurement is physically realised. The estimator's position state is the ARP, because the receiver PCO/PCV it applies are referenced to the ARP. The antenna model covers the step from ARP to phase centre, so only the step from marker to ARP belongs to this document. Treating a marker coordinate as an ARP coordinate, or the reverse, puts a constant bias equal to the eccentricity into the solution, mostly in height.

The leg itself is the RINEX `ANTENNA:DELTA H/E/N` eccentricity $\boldsymbol{e}_{\mathrm{HEN}} = (\delta H,\, \delta E,\, \delta N)^{\mathrm{T}}$, defined as the offset from the marker to the ARP in the topocentric frame *at the marker*. RINEX writes the triple as $(H, E, N) = (\mathrm{Up}, \mathrm{East}, \mathrm{North})$. The rotation of (7) expects (E, N, U), so the triple must be reordered before it is rotated (`Coordinates::markerToArp`, the single owner of this convention):

$$\boxed{\boldsymbol{r}_{\mathrm{ARP}} \;=\; \boldsymbol{r}_{\mathrm{mkr}} \;+\; \boldsymbol{R}_{\mathrm{ENU}}\big(\varphi_{\mathrm{mkr}},\, \lambda_{\mathrm{mkr}}\big) \begin{bmatrix} \delta E \\ \delta N \\ \delta H \end{bmatrix}}, \tag{9}$$

with $(\varphi_{\mathrm{mkr}}, \lambda_{\mathrm{mkr}})$ recovered from $\boldsymbol{r}_{\mathrm{mkr}}$ by the inverse map of (5). Write $\boldsymbol{e}'_{\mathrm{HEN}}$ for the permuted $(\delta E, \delta N, \delta H)^{\mathrm{T}}$ that the rotation acts on.

The inverse is not a sign flip. Reducing an ARP estimate to the monument is not the negative of (9). The local frame is defined at the marker, and the marker is the unknown. Evaluating $\boldsymbol{R}_{\mathrm{ENU}}$ at the ARP instead leaves an error of order $\|\boldsymbol{e}_{\mathrm{HEN}}\|^2 / R_{\oplus}$. The engine therefore takes one fixed-point step (`Coordinates::arpToMarker`):

$$\boldsymbol{r}^{(0)} = \boldsymbol{r}_{\mathrm{ARP}} - \boldsymbol{R}_{\mathrm{ENU}}\big(\mathrm{llh}(\boldsymbol{r}_{\mathrm{ARP}})\big)\, \boldsymbol{e}'_{\mathrm{HEN}}, \qquad \boldsymbol{r}_{\mathrm{mkr}} = \boldsymbol{r}_{\mathrm{ARP}} - \boldsymbol{R}_{\mathrm{ENU}}\big(\mathrm{llh}(\boldsymbol{r}^{(0)})\big)\, \boldsymbol{e}'_{\mathrm{HEN}}, \tag{10}$$

which closes the round trip with (9) to well below the millimetre level this reference grades at.

Where the reduction is applied, and where it is not. Every position derived from a RINEX *header* is reduced: the post-processing filter seed (`Application::initializePosition` and the `PppModel` state initialiser), the real-time seed (`RealTimeEngine::initialize`), the baseline rover a priori, and the closed-loop consumer truth all take the marker through (9), some calling `Coordinates::markerToArp` directly, others through the accessor `RinexParser::getArpPositionXyz` that pairs the two header fields. A coordinate resolved from a Station Information File is a monument coordinate too and is reduced through the same function (the SIF section of the Reference Frames document). A coordinate supplied *directly in the configuration* is not reduced: it is taken at face value, on the stated policy that the user owns its reference point. So the baseline *reference* coordinate is used as given while the rover a priori read from its header is reduced, an asymmetry inside one mode. The network estimator's station coordinates (`StationConfig::position_ecef`, optionally SIF-resolved under `network.use_sif_coordinates`) are monument coordinates held *fixed*, their three position columns dropped from the normal equations. With `network.station_marker_to_arp` (default off; on in both reference configurations, `config/golden/golden_field_refound.json` and `golden_sim_refound.json`) the network setup

reduces each roster coordinate through (9) once, with the station's own RINEX eccentricity, before the coordinate reaches the state, the model or any geometry; the test station's truth coordinate takes the same function. A station whose header carries no eccentricity record is left unchanged and named in a warning. With the setting off, a monument-referenced coordinate is not absorbed by a position state; it survives into the residuals and hence into the estimated clocks, biases, and ionosphere. A zero or absent eccentricity makes (9) an exact, bitwise no-op, which is what keeps header-less RTCM/SBF streams and `0/0/0` stations untouched; the ordering, sign, no-op, and round-trip properties are pinned by the `AntennaEccentricityTest` suite.

1.3 GNSS time scales and the GPS reference epoch

Time inside the engine is carried as a `GnssTime` {week, tow} pair, an integer GPS week and a floating-point time-of-week in seconds, and is expanded to a single continuous count $T = \text{week} \times 604800 + \text{tow}$ wherever a monotone scalar is needed (e.g. ERP interpolation, Section 1.9). A week boundary is 604 800 s and a day 86 400 s (`src/core/math/TimeUtils.cpp`, header constants `SECONDS_IN_WEEK`, `SECONDS_IN_DAY`). The common origin is the GPS epoch, 1980-01-06 00:00:00 UTC, whose Julian Date is fixed in the code:

$$\mathrm{JD}_{\mathrm{GPS}} = 2444244.5 \qquad (\texttt{src/core/math/TimeUtils.cpp}, \texttt{GPS_EPOCH_JD}). \tag{11}$$

GPS time is a continuous atomic scale, since it never inserts a leap second, so it sits a fixed integer offset below International Atomic Time and a fixed fractional offset below Terrestrial Time (`src/core/math/TimeUtils.cpp`, header constants `TAI_GPS_OFFSET`, `TT_GPS_OFFSET`):

$$\boxed{\mathrm{TAI} - \mathrm{GPS} = 19\ \mathrm{s}, \qquad \mathrm{TT} - \mathrm{GPS} = 51.184\ \mathrm{s} = (32.184 + 19)\ \mathrm{s}.} \tag{12}$$

The 32.184 s in the TT relation is the constant TT−TAI offset; together these let any GPS time be promoted to the TT scale on which precession and nutation are evaluated. Terrestrial Time relative to the J2000.0 epoch is measured in Julian centuries,

$$t = \frac{\mathrm{JD}_{\mathrm{TT}} - 2451545.0}{36525.0}, \tag{13}$$

the dimensionless argument of the astronomical series of Sections 1.7 and 1.10 (`TimeUtils::jdToJ2000Centuries`, constants `J2000_JD`, `DAYS_PER_JULIAN_CENTURY`).

Gregorian↔Julian Date. The calendar conversions follow the Meeus algorithm [2]. The forward map, with the month shifted so January and February count as months 13 and 14 of the prior year and a Gregorian-reform correction $b = 2 - A + \lfloor A/4 \rfloor$ for $A = \lfloor y/100 \rfloor$, is (`TimeUtils::gregorianToJulianDay`):

$$\mathrm{JD} = \lfloor 365.25\,(y + 4716) \rfloor + \lfloor 30.6001\,(m + 1) \rfloor + d + b - 1524.5 + \frac{(\text{time of day})}{86400}, \tag{14}$$

and the inverse (`TimeUtils::julianDayToGregorian`) reverses it through the integer auxiliaries $z = \lfloor \mathrm{JD}{+}0.5 \rfloor$ and the Gregorian-reform branch at $z \geq 2299161$. The code keeps two kinds of calendar date apart. A date already on the GPS scale needs no leap correction (`gregorianGpsToGpsTime`). A date on UTC must have the leap seconds at that epoch added to reach GPS time (`TimeUtils::gregorianToGpsTime`). Confusing the two produces whole-second timing errors.

Elapsed time and the start-offset comparison. Epoch times come from a calendar-to-GPS conversion that carries about 2×10^{-5} s of double-precision residue, and a continuous GPS second count near 1.4×10^{9} resolves only 2.4×10^{-7} s. Both are far below a millisecond and far below any epoch schedule, so elapsed time between two epochs is taken on a whole-millisecond grid (`TimeUtils::elapsedMs`):

$$\Delta_{\mathrm{ms}} \;=\; \mathrm{round}\big(10^{3}\,(T_{\mathrm{later}} - T_{\mathrm{earlier}})\big). \tag{15}$$

The start-offset seek (`network.start_offset_hours`) and the banding of the elapsed hour read it. An epoch has reached an offset o when $\Delta_{\mathrm{ms}} \geq \mathrm{round}(10^{3}\,o)$ (`TimeUtils::offsetReached`). An offset that lies on an epoch (a bare hour on a 30 s schedule) is therefore reached by that epoch, where a floating-point comparison reached it one epoch late whenever the residue was negative. An offset between two epochs (a half-epoch back-off) selects the same epoch under either comparison, because no epoch lies within a millisecond of it.

1.4 Leap seconds and per-system dispatch

UTC is tied to GPS by the historical table of leap-second insertions. The engine stores the table as $\big(\mathrm{JD}_k,\, \Delta_{\mathrm{GU}}(k)\big)$ pairs: the Julian Date of 0 h *UTC* on the insertion day and the GPS−UTC offset that holds from then on, the first row being the GPS epoch itself with offset zero (`src/core/math/TimeUtils.cpp`, `LEAP_TABLE`). The dates are UTC, so a lookup compares on the scale of its argument. A date already on UTC compares directly, and a reverse scan returns the offset of the latest boundary not after the epoch:

$$\Delta_{\mathrm{GU}}(\mathrm{JD}_{\mathrm{UTC}}) = \Delta_{\mathrm{GU}}(k^{\star}), \qquad k^{\star} = \max\{\, k : \mathrm{JD}_k \leq \mathrm{JD}_{\mathrm{UTC}} \,\}, \tag{16}$$

falling back to zero before the epoch (`TimeUtils::leapSecondsAtUtcJd`, used by the calendar UTC-to-GPS conversion `TimeUtils::gregorianToGpsTime`). An instant on the GPS scale is moved onto UTC before the comparison. A new offset applies when UTC reaches 0 h of the insertion date, and that instant lies $\Delta_{\mathrm{GU}}(k)$ seconds later on the GPS clock, so

$$k^{\star} = \max\left\{\, k : \mathrm{JD}_{\mathrm{GPS}} - \tfrac{\Delta_{\mathrm{GU}}(k)}{86400} \;\geq\; \mathrm{JD}_k \right\} \tag{17}$$

(`TimeUtils::leapSecondsGpsUtc`). Comparing a GPS-scale date with the UTC table directly would apply each offset early by its own value (18 s at the 2017-01-01 insertion). TAI−UTC is then $\Delta_{\mathrm{GU}} + 19$ s from (12) (`TimeUtils::leapSecondsTaiUtc`). The lookup must depend on the epoch. A leap second was inserted between 2005 and 2010, so the same UTC calendar instant maps to different GPS times in those two years.

The wall-clock leap constant and its runtime override. The boundary-aware table of (16) is the authority for every *epoch-referenced* conversion (a dated calendar timestamp, GLONASS day arithmetic, UTC formatting) because each of those carries the epoch at which the correct offset is known. The one conversion that does not is the promotion of the *system wall clock* to GPS time (`TimeUtils::nowContinuousGpsSeconds`, `nowGpsTime`), used only on the real-time decode and latency-stamping paths: POSIX `system_clock` is UTC and carries no epoch of its own, so the offset cannot be looked up and must instead be supplied as a fixed count. That count is the compiled constant `TimeUtils::LEAP_SECONDS` = 18 s (valid from 2017-01-01), read through an atomic accessor (`currentLeapSeconds`) so that a future insertion can be applied to a live binary without a rebuild: `setLeapSecondsOverride`, driven by the config key `realtime.leap_seconds` (`src/core/config/GnssConfigLoad.cpp`), replaces it at real-time startup. The override deliberately touches *only* the

wall-clock path; the epoch-dependent (16) and (17) lookups are left alone, since they are correct at every historical epoch, the insertion instants included, and an override there would corrupt rather than update them.

Per-constellation Gregorian dispatch. The four supported time systems differ only in how a RINEX/broadcast calendar timestamp is referred to the GPS scale, dispatched by a system tag (`TimeUtils::GnssTimeFromDate`):

- **GPS / Galileo**: already on the GPS scale; the timestamp is converted with no leap correction (GST is steered to GPS time and shares the same week/second framing to within the few nanoseconds the ISB state of the State Vector document absorbs).
- **BeiDou**: the timestamp is on BDT, which runs 14 s behind GPS (Section 1.6); the code converts on the GPS-scale assumption and then *adds* 14 s, carrying any week roll-over (`TimeUtils::GnssTimeFromDate`, BeiDou branch).
- **GLONASS**: the RINEX timestamp is UTC; the full leap correction of (16) is applied to bring it onto GPS time (`TimeUtils::GnssTimeFromDate`, GLONASS branch, through `TimeUtils::gregorianToGpsTime`).

GPS and Galileo therefore carry no leap correction, BeiDou a fixed 14 s, and GLONASS the full epoch-dependent leap count. Each matches the time reference that defines the constellation. There is a fourth branch: an unrecognised time-system tag falls through to the GPS-scale conversion with only a trace-level note, and a calendar date that fails validation returns the sentinel $\{0, 0\}$ rather than a fabricated epoch (`TimeUtils::GnssTimeFromDate`). The fallback logs nothing at default verbosity, so an unexpected tag produces a timing error of the full leap count with no warning.

1.5 GLONASS time-of-day to full GPS week

GLONASS broadcast and SSR records frequently carry only a time-of-day, seconds since the start of the GLONASS day, with no week and no absolute date. GLONASS time runs on UTC(SU) advanced by three hours (Moscow), so its day boundary is three hours ahead of the UTC midnight. Reconstructing a full GPS week from a bare time-of-day therefore needs an *anchor*: an approximate GPS-seconds estimate of the true epoch (typically the current obs epoch), against which the day and the ± 12 h wrap are resolved (`TimeUtils::glonassTodToGpsTime`, after RTKLIB's `adjday_glot`). The procedure, with day length $D = 86\,400$ s, half-day $D/2$, and Moscow offset $M = 3 \times 3600$ s:

1. Convert the anchor GPS seconds to UTC by subtracting the leap count at that epoch (the GPS-scale lookup (17)), $t_{\mathrm{UTC}} = t_{\mathrm{anchor}} - \Delta_{\mathrm{GU}}$, then shift into the Moscow-day frame $t_{\mathrm{M}} = t_{\mathrm{UTC}} + M$ and locate that day's midnight $t_0 = \lfloor t_{\mathrm{M}}/D \rfloor\, D$, giving the anchor's time-of-day $\tau_{\mathrm{a}} = t_{\mathrm{M}} - t_0$.
2. Place the reported time-of-day τ in the anchor's day, resolving the midnight wrap toward the nearest match:

$$\tau \leftarrow \begin{cases} \tau + D, & \tau < \tau_{\mathrm{a}} - D/2, \\ \tau - D, & \tau > \tau_{\mathrm{a}} + D/2, \\ \tau, & \text{otherwise}, \end{cases} \qquad t_{\mathrm{GPS}} = \underbrace{(t_0 + \tau)}_{\text{Moscow}} - M + \Delta_{\mathrm{GU}}. \tag{18}$$

The last expression removes the Moscow offset to recover UTC and then adds the same leap count Δ_{GU} to return to the GPS scale. The leap term enters once with each sign and cancels. It appears

only because the Moscow day must be located on the UTC scale. If no usable anchor is supplied (a non-positive value) the routine returns the sentinel $\{0, 0\}$ rather than fabricate a week, so the caller can detect the failure and fall back; the continuous-seconds-to-week split that closes the routine carries the standard negative-TOW correction (`gpsSecondsToGnssTime`).

1.6 BeiDou time: the 14 s offset and independent week base

BeiDou time (BDT) differs from GPS time by a fixed number of seconds and by an independent week numbering. Different modules handle the two parts. The *seconds* relation,

$$\mathrm{BDT} = \mathrm{GPS} - 14\ \mathrm{s}, \tag{19}$$

reflects that BDT was synchronised to UTC at its 2006 zero epoch, by which time GPS already led UTC by 14 s; it is applied in `TimeUtils` when a BeiDou calendar timestamp is converted to GPS time (`TimeUtils::GnssTimeFromDate`, BeiDou branch; Section 1.4). The *week* relation is separate. BDT counts weeks from its own zero epoch (2006-01-01), which is GPS week 1356, and this base is applied at query time by the ephemeris store, not by `TimeUtils` (the file-local constant `kBdtToGpsOffsetSec`, consumed by `LiveEphemerisStore::computeSatellitePV`). When the store lifts a BeiDou ephemeris reference time onto the common GPS scale for a cross-system age check it adds the combined offset

$$\boxed{\Delta_{\mathrm{BDT}\to\mathrm{GPS}} = 1356 \times 604800\ \mathrm{s} + 14\ \mathrm{s},} \tag{20}$$

the week base plus the 14 s of (19) (`kBdtToGpsOffsetSec`, applied by the file-local `toeGpsContinuous`). Without the week base every BeiDou ephemeris would appear about 1356 weeks ($\sim$26 years) old, and the staleness guard in `LiveEphemerisStore::computeSatellitePV` would reject it without a message. The offset does not cancel, because the query time is on the GPS scale while the stored t_{oe} uses the BDT week.

1.7 Legacy ECI→ECEF: the equinox-based chain

The Sun, the Moon, and for some product streams the satellite positions are given in a quasi-inertial celestial frame. The tides, attitude, and phase wind-up that use them are computed in the terrestrial ECEF frame. The Earth-orientation matrix $\boldsymbol{U}$ connects the two: $\boldsymbol{r}_{\mathrm{ECEF}} = \boldsymbol{U}\,\boldsymbol{r}_{\mathrm{ECI}}$. The default (always-available) path is the classical equinox-based chain following the IAU-1976 precession and IAU-1980 nutation conventions [4], composed as (`Planetary::eci2ecef`):

$$\boxed{\boldsymbol{U} = \boldsymbol{W}_{\mathrm{pm}}\ \boldsymbol{R}_3(\mathrm{GAST})\ \boldsymbol{N}_{\mathrm{nut}}\ \boldsymbol{P}_{\mathrm{prec}}.} \tag{21}$$

Here $\boldsymbol{R}_1, \boldsymbol{R}_2, \boldsymbol{R}_3$ are the elementary right-handed axis rotations of `Planetary::Rxyz` and each factor reverses one piece of the celestial-to-terrestrial orientation: precession and nutation orient the true equator and equinox of date, the sidereal rotation $\boldsymbol{R}_3(\mathrm{GAST})$ spins the frame about the pole by the apparent sidereal time, and polar motion $\boldsymbol{W}_{\mathrm{pm}}$ ties the celestial pole to the conventional terrestrial pole.

Precession (IAU 1976). With t the TT centuries of (13) and arcsecond coefficients converted to radians, the three equatorial precession angles ζ_A, θ_A, z_A and the mean obliquity ε_0 are (the

precession block of `Planetary::eci2ecef`):

$$\begin{aligned}\zeta_A &= 2306.2181\,t + 0.30188\,t^2 + 0.017998\,t^3,\\ \theta_A &= 2004.3109\,t - 0.42665\,t^2 - 0.041833\,t^3,\\ z_A &= 2306.2181\,t + 1.09468\,t^2 + 0.018203\,t^3,\\ \varepsilon_0 &= 84381.448 - 46.8150\,t - 0.00059\,t^2 + 0.001813\,t^3 \quad \text{(arcsec)},\end{aligned} \tag{22}$$

assembled as $\boldsymbol{P}_{\text{prec}} = \boldsymbol{R}_3(-z_A)\,\boldsymbol{R}_2(\theta_A)\,\boldsymbol{R}_3(-\zeta_A)$.

Nutation (IAU 1980, 106 terms). The nutation in longitude $\Delta\psi$ and in obliquity $\Delta\varepsilon$ are summed over the 106-term IAU-1980 series [4] in the five Delaunay fundamental arguments $f = (l, l', F, D, \Omega)$ (`Planetary::nutIau1980`; arguments from `Planetary::astArgs`):

$$\Delta\psi = 10^{-4}\sum_{i=1}^{106}\big(A_i^{\psi} + \dot{A}_i^{\psi}\,t\big)\sin\big(\boldsymbol{n}_i\cdot f\big), \quad \Delta\varepsilon = 10^{-4}\sum_{i=1}^{106}\big(A_i^{\varepsilon} + \dot{A}_i^{\varepsilon}\,t\big)\cos\big(\boldsymbol{n}_i\cdot f\big), \tag{23}$$

in arcseconds (the 10^{-4} scales the tabulated 0.1 mas units), then converted to radians. The nutation matrix rotates by the true obliquity $\varepsilon_0 + \Delta\varepsilon$: $\boldsymbol{N}_{\text{nut}} = \boldsymbol{R}_1(-\varepsilon_0 - \Delta\varepsilon)\,\boldsymbol{R}_3(-\Delta\psi)\,\boldsymbol{R}_1(\varepsilon_0)$ (the nutation block of `Planetary::eci2ecef`).

Apparent sidereal time and the 0h-UT1 reference. Greenwich Apparent Sidereal Time adds the equation of the equinoxes, the nutation in longitude projected on the equator, plus the two leading complementary terms in the lunar node Ω, to the mean sidereal time (the GAST block of `Planetary::eci2ecef`):

$$\text{GAST} = \text{GMST} + \Delta\psi\cos\varepsilon_0 + \big(0.00264\sin\Omega + 0.000063\sin 2\Omega\big)\ \text{arcsec}. \tag{24}$$

GMST itself is built from the IAU-1982 model evaluated at the *start* of the current UT1 day plus the Earth's rotation through the elapsed day fraction (`Planetary::utc2gmst`). The reference instant must be 0h UT1 of the calendar day, and because a Julian Date is referenced to *noon*, that midnight is recovered as

$$\text{JD}_{0\text{h}} = \big\lfloor \text{JD}_{\text{UT1}} - 0.5 \big\rfloor + 0.5, \tag{25}$$

not $\lfloor \text{JD} \rfloor$ (same routine). The bare floor gives the *previous* noon for any epoch after midnight. That half-day error is roughly 0.5° in GMST and, through (21), of order 10^6 km in the rotated Sun position. A comment in `Planetary::utc2gmst` records this failure.

Polar motion. The terrestrial-pole tie is $\boldsymbol{W}_{\text{pm}} = \boldsymbol{R}_2(-x_p)\,\boldsymbol{R}_1(-y_p)$ for the interpolated pole coordinates (x_p, y_p) in radians (the polar-motion block of `Planetary::eci2ecef`), the small-angle two-axis rotation that aligns the celestial intermediate pole of date with the IERS conventional pole.

1.8 CIO-based ITRS↔GCRS (IAU 2000A/2006)

An opt-in path (config key `estimation.enable_cio_frame`, getter `GnssConfig::enableCioFrame`; **default off**, so the equinox chain of (21) is the shipped path) replaces the equinox chain with the modern Celestial Intermediate Origin (CIO) based transform of the IAU 2000A/2006 conventions [4], evaluated through SOFA [1] and mirroring the M-PAGES `NgsFormC2T` construction (`Planetary::celestialToTerrestrial`). The celestial-to-terrestrial matrix is composed as

$$\boxed{\boldsymbol{U} = \texttt{POM}(x_p, y_p, s')\;\boldsymbol{R}_3(\theta_{\text{ERA}})\;\texttt{C2I}(X, Y, s),} \tag{26}$$

i.e. the SOFA composition `iauC2tcio`. The Celestial Intermediate Pole coordinates (X, Y) and the CIO locator s come from the full IAU-2000A precession-nutation model via `iauXys00a`, after which the *observed* celestial-pole offsets are injected additively,

$$X \leftarrow X + \mathrm{d}X, \qquad Y \leftarrow Y + \mathrm{d}Y, \tag{27}$$

and the celestial-to-intermediate matrix is formed by `iauC2ixys`. The diurnal rotation is the Earth Rotation Angle θ_{ERA} (a strictly linear function of UT1, `iauEra00`) rather than the sidereal time of (24), and the terrestrial-pole tie `POM` uses `iauPom00` with the TIO locator s' from `iauSp00`. Two-part Julian Dates are passed with the $(2400000.5, \mathrm{MJD})$ split for numerical conditioning (same routine). The two paths should agree. With $\mathrm{d}X = \mathrm{d}Y = 0$ the CIO result is the bare IAU-2000A model. It differs from the legacy equinox chain only by the frame bias and the model version, which is below a milliarcsecond. The engine checks this at startup by logging the angular separation of the two Sun directions (the startup diagnostic in `Planetary::computeSunMoonPosition`).

1.9 Earth-orientation parameters: parse and interpolation

Both transforms above need the time-varying Earth-orientation parameters $(x_p, y_p, \mathrm{UT1} - \mathrm{UTC}, \mathrm{d}X, \mathrm{d}Y)$. These arrive in a standard IGS `.ERP` file, parsed by `ProductParser::parseErp` (`src/core/products/ProductParser.cpp`). The parser reads the MJD time tag and the pole/UT1 columns and applies the IGS scale factors: the pole coordinates are tabulated in 10^{-6} arcseconds and UT1−UTC in 10^{-7} seconds (`ProductParser::parseErp`):

$$x_p = (\mathrm{col}) \times 10^{-6}\,\mathrm{arcsec}, \quad y_p = (\mathrm{col}) \times 10^{-6}\,\mathrm{arcsec}, \quad (\mathrm{UT1} - \mathrm{UTC}) = (\mathrm{col}) \times 10^{-7}\,\mathrm{s}. \tag{28}$$

The MJD tag is converted to a GPS week/TOW relative to the GPS-epoch MJD (same routine). The CIP offsets $\mathrm{d}X, \mathrm{d}Y$ are *not* carried by the standard `.ERP` format and default to zero; because the IAU-2000A model of (26) already includes the modelled precession-nutation, $\mathrm{d}X = \mathrm{d}Y = 0$ is the correct nominal value, not a missing input (same routine). The parser does no interpolation. `Planetary::interpolateErp` binary-searches the chronologically sorted table on continuous GPS seconds and linearly interpolates every component between the bracketing entries:

$$p(t) = p_0 + \frac{t - t_0}{t_1 - t_0}\,(p_1 - p_0), \qquad p \in \{x_p, y_p, \mathrm{UT1}{-}\mathrm{UTC}, \mathrm{d}X, \mathrm{d}Y\}, \tag{29}$$

clamping to the nearest endpoint outside the table span (same routine). The interpolated (x_p, y_p) feed the polar-motion factor of (21) / (26), while UT1−UTC corrects UTC to UT1 before the sidereal/ERA evaluation. The product-evaluation document (Product Evaluation) cross-references this section rather than re-deriving the polar-motion application, since the same $\boldsymbol{W}_{\mathrm{pm}}$ rotates both the luni-solar vectors and any celestial-frame satellite product.

1.10 The Sun and Moon series these transforms position

The orientation chain exists to place the Sun and Moon in ECEF for the geophysical and observation-model corrections. The celestial-frame positions are computed first, from an analytic series in the J2000 centuries t and the Delaunay arguments f (`Planetary::sunMoonPosEci`), an enhanced RTKLIB/Meeus model [2, 3]. The Sun's ecliptic longitude is the mean longitude plus the equation of centre (three harmonics of the solar mean anomaly $M_\odot$) and small Venus, Jupiter, and lunar perturbation terms (same routine); the obliquity rotation

$$\varepsilon = 23.439291 - 0.0130042\,t \quad (\mathrm{deg}) \tag{30}$$

carries the resulting ecliptic $(\lambda_\odot, \beta_\odot)$ into equatorial ECI components:

$$\begin{aligned} r_x &= r\,\cos\beta_\odot \cos\lambda_\odot, \\ r_y &= r\,(\cos\varepsilon\,\cos\beta_\odot \sin\lambda_\odot - \sin\varepsilon\,\sin\beta_\odot), \\ r_z &= r\,(\sin\varepsilon\,\cos\beta_\odot \sin\lambda_\odot + \cos\varepsilon\,\sin\beta_\odot), \end{aligned} \tag{31}$$

with the solar distance r from the three-harmonic radius series. The Moon's longitude, latitude, and horizontal-parallax-derived distance follow the analogous Brown-series truncation in the same five fundamental arguments (same routine). The resulting ECI vectors are then rotated to ECEF by exactly the $\boldsymbol{U}$ of (21) or (26) according to the `estimation.enable_cio_frame` toggle (default off, Section 1.8), in `Planetary::computeSunMoonPosition`; the ECEF Sun and Moon are the inputs consumed by the solid-Earth and ocean tides of the Site Displacement document and by the satellite-attitude and phase-wind-up models of the Observation Physics document. The two luni-solar series are ∼100 km accurate in position. Their output is in the mean-of-date frame, not J2000, and the ECI-to-ECEF rotation of the legacy chain applies precession again, so on the default path the Sun and Moon directions carry the precession accumulated since J2000 a second time (about 50 arcsec per year, roughly a third of a degree in 2024). `estimation.enable_sun_frame_correction` (default off, byte-identical when off) de-precesses the of-date vectors to J2000 before the rotation and removes that error (`Planetary::precessionIau1976`). The same setting makes the satellite attitude model form the orbit normal from the inertial velocity (the attitude section of the Observation Physics document). A non-finite guard substitutes a nominal 1 AU / mean-lunar-distance fallback rather than propagate a NaN into the correction stack (the output-validation tail of `Planetary::computeSunMoonPosition`).

References

[1] IAU SOFA Board. *SOFA Tools for Earth Attitude*. Software version 18; http://www.iausofa.org/. International Astronomical Union, Standards of Fundamental Astronomy (SOFA). 2021.

[2] Jean Meeus. *Astronomical Algorithms*. Richmond, VA, USA: Willmann-Bell, 1991. ISBN: 0-943396-35-2.

[3] Oliver Montenbruck and Eberhard Gill. *Satellite Orbits: Models, Methods and Applications*. Berlin, Heidelberg: Springer, 2000. DOI: 10.1007/978-3-642-58351-3.

[4] Gérard Petit and Brian Luzum. *IERS Conventions (2010)*. IERS Technical Note 36. International Earth Rotation and Reference Systems Service (IERS). Frankfurt am Main: Verlag des Bundesamts für Kartographie und Geodäsie, 2010. ISBN: 3-89888-989-6.